LUCASNTT FOUNDATIONS

A PREPRINT

Guillaume Pierre Hérault
Geneva, Switzerland

December 22, 2019

ABSTRACT

This document explains with simple words and simple math concepts the basic foundations of the following concepts: Discrete Fourier Transforms, FFT, NTT, convolutions, and large multiplications.

Contents

I Theoretical Foundations of LucasNTT 4

1 Introduction 5

2 A Simple Transform 5

3 Discrete Fourier Transform (DFT) 6

3.1 Taxonomy . . . 6

3.2 Definition . . . 6

3.3 Pseudocode of the DFT . . . 8

3.4 Math Papers . . . 8

3.5 Matrix notation . . . 9

3.6 Inverse Discrete Fourier Transform . . . 9

4 Roots of unity 10

4.1 in Complex-field . . . 10

4.2 Example in Complex-field . . . 11

4.2.1 First step of the example: forward DFT . . . 11

4.2.2 Second step of the example: square . . . 11

4.2.3 Third step of the example: Inverse DFT . . . 12

4.3 Another notation . . . 12

5 Modular Arithmetic 13

6 NTT 13

6.1 Definition . . . 13

6.2 Example in $\mathbb{Z}_p$ . . . 14

6.2.1 First step of the example: forward DFT . . . 14

6.2.2 Second step of the example: square . . . 15

6.2.3 Third step of the example: Inverse DFT . . . 15

6.3 Finding the inverse in $\mathbb{Z}_p$ . . . 16

6.4 Finding the primitive N^{th} root of unity modulo p . . . 16

6.5 Choosing the perfect NTT . . . 17

6.5.1 The early papers and the Chinese Remainder Theorem (CRT) . . . 18

6.5.2 Mersenne Number Transforms . . . 18

6.5.3 Fermat Number Transforms . . . 18

6.5.4 Moduli of the form $2^j - 2^k + 1$ . . . 19

6.5.5 Other moduli . . . 19

6.6 The modulus $p = 2^{64} - 2^{32} + 1$ . . . 19

7 Convolutions **19**
7.1 Acyclic convolutions . . . 20
7.2 Acyclic convolutions . . . 20
7.3 Convolution theorem . . . 21

8 Multiplication Algorithm for large integers **21**
8.1 Evaluating the length of the DFT . . . 22
8.2 Initiating the DFT multiplication . . . 22
8.3 The DFT multiplication algorithm . . . 22

9 Multiplication Algorithm modulo q **22**

10 IBDWT **22**

11 FFT **22**

II LucasNTT Implementation **23**

12 Arithmetics **24**
12.0.1 Arithmetic in this field . . . 24

13 A trier **25**

14 Headings: first level **26**
14.1 Headings: second level . . . 26
14.1.1 Headings: third level . . . 26

15 Examples of citations, figures, tables, references **26**
15.1 Figures . . . 27
15.2 Tables . . . 27
15.3 Lists . . . 27

Part I
Theoretical Foundations of LucasNTT

1 Introduction

This document is the kind of document I was looking for when I started my journey to the FFT. At the beginning I felt lost among the tons of math papers that claim to have discovered a new way to calculate the FFT. THis document is not only intended for mathematicians. You can be an IT programmer willing to develop your own FFT. You can be a student trying to understand what is a FFT. Maybe you are just curious to understand how dare we can multiply huge numbers with millions of digits in milliseconds. But you don't know where to start. FFT papers are countless. And maybe you find most of them difficult to understand. The objective is to explain those concepts simply.

Nevertheless this document is about math. I have chosen the math language (rather than a writing more IT oriented) because it's a universal language and an opportunity for novices to extend their knowledge. Hopefully to understand this document you don't need to be an expert. You just need to remember some notions learnt at secondary/high school, meaning around the ages of 17 and 19 in most of the countries. For example we will speak about polynomial functions, complex numbers and modular arithmetic. Sometimes there may be some terms that you don't fully understand. This is for the sake of precision and to direct you in the right path in case you want to improve your knowledge. However it should not prevent you to understand the global meaning.

In short: simple math language, no proof and plenty of examples

2 A Simple Transform

This section demonstrates how a simple transform can simplify the multiplication. Let us start with a polynomial function of degree 3 in the real domain, for example

$$f(x) = 1 + x + x^2 + x^3 \tag{1}$$

A shorter way to write $f(x)$ is to store the coefficients into a structure and this leads to the following sequence: $\{1, 1, 1, 1\}$. Let us call that the "coefficient-view". It is a "view" in the sense that this sequence $\{1, 1, 1, 1\}$ is another mode of writing but still uniquely defines $f(x)$.
Now we evaluate this polynomial function for 4 distinct values, for example $x = -1$, $x = 0$, $x = 1$, and $x = 2$. The results are $0, 1, 4, 15$. These 4 points $(-1, 0), (0, 1), (1, 4)$ and $(2, 15)$ also uniquely identify this function, meaning that there is only one polynomial function of degree 3 that joins these 4 points, and it's $f(x)$.

In other words, the sequence $\{0, 1, 4, 15\}$ is an alternate view of $f(x)$ at the condition that we previously agree on the values to be evaluated (if we always use the same distinct values as input). Let us call that alternate view the "point-value-view". At this stage we have defined a "Transform" that takes a 4-length sequence in coefficient-view as input, and outputs another sequence in point-value-view.

{1, 1, 1, 1}	$\Longrightarrow$	{0, 1, 4, 15}
coefficient-view		point-value-view

This transform just changes the point of view, but does not change the underlying polynomial function. And this transform is invertible because you can reconstruct the original polynomial function from the point-value-view. Please note that this backward transform will not be detailed here because it's not the focus (see Lagrange polynomial interpolation).
Now the interesting part: we will use this transform to facilitate the multiplication of polynomials. For the sake of simplicity, let's say we want to multiply $f(x)$ by himself:

$$g(x) = f(x) \times f(x) \tag{2}$$

Finding $g(x)$ using the coefficient-view is boring. You have to repeatedly use the distributive law for each coefficient and it requires 16 multiplications plus 9 additions. With some concentration you can find the expected result:

$$g(x) = 1 + 2x + 3x^2 + 4x^3 + 3x^4 + 2x^5 + x^6 \tag{3}$$

But using the point-value-view, the multiplication is much easier. By definition, $g(0) = f(0) \times f(0)$, $g(1) = f(1) \times f(1)$, $g(2) = f(2) \times f(2)$ and so on. So given the sequence in point-value-view, you just have to square once each item of the sequence. As the input sequence is $\{0, 1, 4, 15\}$, the calculated result is $\{0, 1, 16, 225\}$. In this world, the multiplication is faster.

The only issue is that the expected degree of g is 6, therefore you will need 7 points to uniquely identify g. Back to the beginning, you should have evaluated initially f for 7 distinct values instead of 4. Let's take for example the following values:

$$f(-3) = -20 \qquad f(-2) = -5 \qquad f(-1) = 0 \qquad f(0) = 1 \qquad f(1) = 4 \qquad f(2) = 15 \qquad f(3) = 40$$

Each of these values will be squared one by one, meaning 7 multiplications instead of 16. Of course you probably need to get the final result in the standard coefficient-view, so you have to apply a backward transform. In final you have the following steps:

{1, 1, 1, 1, 0, 0, 0} coefficient-view	$\Longrightarrow$ forward transform	{-20, -5, 0, 1, 4, 15, 40} point-value-view
		$\Downarrow$ square
{1, 2, 3, 4, 3, 2, 1} coefficient-view	$\Longleftarrow$ backward transform	{400, 25, 0, 1, 16, 225, 1600} point-value-view

Indeed $1, 2, 3, 4, 3, 2, 1$ are the expected coefficients of $g(x)$, see (3).

In summary (what we will call later) the acyclic convolution of f by itself is obtained by this 3 steps algorithm: forward transform, pointwise-square, then backward transform. The complexity of the overall algorithm is mainly reduced to the complexity of transforming a sequence forward and backward.

Please note that to anticipate the size of the expected result, we have considered $f(x)$ as a polynomial of degree 6 with zero coefficient for x^4, x^5 and x^6. This explains the zeros in the input sequence $\{1, 1, 1, 1, 0, 0, 0\}$. In other words the input sequence has been "zero-padded" by appending zeros, in order to avoid overflow when squaring.

In this section, we learnt that some transforms can be useful to address a problem in a different manner. This example also contains the keys to understand:

- The Discrete Fourier Transform
- The Convolution theorem
- The multiplication algorithm of large integers using Transforms
- Why sometimes we zero-pad the input sequence
- Acyclic and cyclic convolutions

Also please note that the algorithm explained above is very close to the Toom-Cook algorithm used to multiply middle-size integers. As we target large-size integers, let us introduce now the Discrete Fourier Transform.

3 Discrete Fourier Transform (DFT)

3.1 Taxonomy

The "Discrete Fourier Transform" is applied on a finite set of elements (this explains the term "Discrete"). It should not be confused with the general infinite Fourier integral transform. The current document does not cover the general "Fourier Transform" but only the DFT. For a comparison of the two transforms, please refer to Gentleman and Sande [1].

3.2 Definition

At this stage we may use some naming conventions. Let N be the length of the input and output sequences. For the rest of this document, N is assumed to be a positive integer. Let A be the input sequence made of N elements a_j.

$$A = \{a_0, a_1, ..., a_{N-1}\} \tag{4}$$

Let us also name the resulting output sequence:

$$\widehat{A} = \{\widehat{a}_0, \widehat{a}_1, ..., \widehat{a}_{N-1}\} \tag{5}$$

In the example of the previous section, we had $N = 7$, $a_0 = a_1 = a_2 = a_3 = 1$ and $a_4 = a_5 = a_6 = 0$, then $\widehat{a}_0 = f(-3) = -20$, $\widehat{a}_1 = f(-2) = -5$ and so on. We have chosen 7 arbitrary integer values for evaluating $f(x)$. Now imagine that there exist much useful values than these ones. Some values that facilitate the backward transform, and open the door to a large variety of optimizations. Let us evaluate $f(x)$ for the following values: w^0, w^1, w^2, ..., w^6 where w will be mysteriously named "primitive N^{th} root of unity".
Then we have

$$\begin{aligned}
\widehat{a}_0 &= f(w^0) = f(1) = a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 \\
\widehat{a}_1 &= f(w^1) = a_0 + a_1 w \ + a_2 w^2 \ + a_3 w^3 \ + a_4 w^4 \ + a_5 w^5 \ + a_6 w^6 \\
\widehat{a}_2 &= f(w^2) = a_0 + a_1 w^2 + a_2 w^4 \ + a_3 w^6 \ + a_4 w^8 \ + a_5 w^{10} + a_6 w^{12} \\
&... \\
\widehat{a}_6 &= f(w^6) = a_0 + a_1 w^6 + a_2 w^{12} + a_3 w^{18} + a_4 w^{24} + a_5 w^{30} + a_6 w^{36}
\end{aligned} \tag{6}$$

We are almost touching the Discrete Fourier Transform (DFT). Now we need to leave the real domain and think about a more generic transform valid in different algebraic domains. And instead of limiting us to $N = 7$, let us write the formulas for any N. Let us simply write the output elements of the DFT as

$$\begin{aligned}
\widehat{a}_0 &= a_0 + a_1 + a_2 + ... + a_{N-1} \\
\widehat{a}_1 &= a_0 + a_1 w \ + a_2 w^2 + ... + a_{N-1} w^{N-1} \\
\widehat{a}_2 &= a_0 + a_1 w^2 + a_2 w^4 + ... + a_{N-1} w^{2(N-1)} \\
&... \\
\widehat{a}_{N-1} &= a_0 + a_1 w^{N-1} + a_2 w^{2(N-1)} + ... + a_{N-1} w^{(N-1)(N-1)}
\end{aligned} \tag{7}$$

Just for convenience, this can be written in a more compact view

$$\begin{aligned}
\widehat{a}_0 &= \sum_{j=0}^{N-1} a_j \\
\widehat{a}_1 &= \sum_{j=0}^{N-1} a_j w^j \\
\widehat{a}_2 &= \sum_{j=0}^{N-1} a_j w^{2j} \\
&... \\
\widehat{a}_{N-1} &= \sum_{j=0}^{N-1} a_j w^{j(N-1)}
\end{aligned} \tag{8}$$

This leads to the following definition

Definition 1. Discrete Fourier Transform (DFT)
Let A be a sequence of N elements a_j that belong to some algebraic domain.

$$A = \{a_0, a_1, ..., a_{N-1}\}$$

The DFT of A is the transformed sequence $\widehat{A} = DFT(A)$ defined by the following N elements

$$\widehat{a}_k = \sum_{j=0}^{N-1} a_j w^{jk} \tag{9}$$

where w is the primitive N^{th} root of unity in the domain.

At this stage the reader may be surprised to see that this transform does not seem to be "fast". Indeed when N grows we need to multiply a great number of times each element of the input sequence. Yes the DFT is not a fast transform. We will see later that the DFT can be optimized with fast algorithms which will be called "Fast Fourier Transforms" (FFT). However, before trying to optimize, one should perfectly know the underlying base algorithm.

3.3 Pseudocode of the DFT

Here is a computer function that returns the DFT of a sequence A. Depending on the chosen domain (complex numbers, finite field...), additional code may be necessary. To be as close as possible to the definition 1, the same variables (`j` and `k`) have been used for the two nested loops.

```
function DFT (A, N, w) {
  var At;
  for (k = 0; k < N; k++) {
    At[k] = 0;
    for (j = 0; j < N; j++) {
      At[k] = At[k] + A[j] * Math.pow(w, j * k);
    }
  }
  return At;
}
```

Please note that the style and syntax used for the pseudo-code in this document is borrowed from Javascript. `Math.pow()` is the function for exponentiation because this notation is very close to the syntax of many programming languages (C++, C#, Java...).

3.4 Math Papers

If you are considering the reading of math papers on DFT and FFT, take attention to the naming conventions. Many math papers on this topic start by defining the DFT with their own words. For example in 1965 James W. Cooley and John W. Tukey published an historic paper named "An Algorithm for the Machine Calculation of Complex Fourier Series" [2] starting with

> Consider the problem of calculating the complex Fourier series
>
> (1) $$X(j) = \sum_{k=0}^{N-1} A(k) \cdot W^{jk}, \qquad j = 0, 1, \cdots, N-1,$$
>
> where the given Fourier coefficients $A(k)$ are complex and W is the principal Nth root of unity,

21 years later the same author James W. Cooley wrote with Ramesh C. Agarwal a paper "Fourier transform and convolution subroutines for the IBM 3090 Vector Facility" [3] with the following words:

> The N-point discrete Fourier transform (DFT) of a vector, $x(n)$, $n = 0, \cdots, N-1$, is the vector defined by
>
> $$y(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk} \qquad k = 0, \cdots, N-1. \qquad (1)$$

When reading these papers, take time to understand the naming conventions of the authors. How do they call the length N, the root of unity w, the items of the sequence $\{a_0, a_1, ..., a_{N-1}\}$. Here the terms *serie* and *vector* are used instead of *sequence*.

3.5 Matrix notation

There is another convenient way to view the DFT using matrices. The formula (7) for $N = 7$ can be written:

$$[\widehat{a}_0 \quad \widehat{a}_1 \quad \widehat{a}_2 \quad \widehat{a}_3 \quad \widehat{a}_4 \quad \widehat{a}_5 \quad \widehat{a}_6] = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & w & w^2 & w^3 & w^4 & w^5 & w^6 \\ 1 & w^2 & w^4 & w^6 & w^8 & w^{10} & w^{12} \\ 1 & w^3 & w^6 & w^9 & w^{12} & w^{15} & w^{18} \\ 1 & w^4 & w^8 & w^{12} & w^{16} & w^{20} & w^{24} \\ 1 & w^5 & w^{10} & w^{15} & w^{20} & w^{25} & w^{30} \\ 1 & w^6 & w^{12} & w^{18} & w^{24} & w^{30} & w^{36} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \end{bmatrix} \tag{10}$$

Indeed the DFT formula can be written as a matrix product:

$$\widehat{A} = TA \tag{11}$$

where the coefficients of T are given by $(T)_{jk} = w^{jk}$

The matrix notation is powerful and compact. It is often used in the FFT papers because it is a way to lead to fast algorithms by factorisation.

By convention we can decide to keep only the exponent of w. Inspired by Pease [4] the middle matrix T then becomes:

$$T_N = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 1 & 2 & 3 & 4 & \dots & N-1 \\ 0 & 2 & 4 & 6 & 8 & \dots & 2(N-1) \\ 0 & 3 & 6 & 9 & 12 & \dots & 3(N-1) \\ 0 & 4 & 8 & 12 & 16 & \dots & 4(N-1) \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & N-1 & 2(N-1) & 3(N-1) & 4(N-1) & \dots & (N-1)^2 \end{bmatrix} \tag{12}$$

3.6 Inverse Discrete Fourier Transform

Now we can explore the inverse (backward) transform:

Definition 2. Inverse Discrete Fourier Transform (IDFT)
Using the same naming conventions as in Definition 1, the inverse transform $A = IDFT(\widehat{A})$ is defined by the following N elements

$$a_j = \frac{1}{N} \sum_{k=0}^{N-1} \widehat{a}_k w^{-jk} \tag{13}$$

This backward transform brings you back to your initial position, meaning that $A = IDFT(DFT(A))$.

In definition 2 there are only two differences with the forward transform:

- We use w^{-jk} instead of w^{jk}
- We scale the elements by $\frac{1}{N}$

The backward transform is so close to the forward transform that the same computer program can be used (under certain conditions) for both cases, as outlined in 1988 by Duhamel [5].

Please note that the scaling factor $\frac{1}{N}$ is sometimes replaced in the literature by $\frac{1}{\sqrt{N}}$ and applied both to the DFT and the IDFT, meaning that it is distributed equally between the two transforms for symmetry reasons. It should also be mentioned that some authors invert the sign of the exponent of the roots of unity (w^{-jk} for the forward transform and w^{jk} for the backward transform).

Definitions 1 and 2 stand for any algebraic domain in which $\frac{1}{N}$ and w both exist. The most common usage is the Complex-field which will be shortly discussed later. But the DFT can also be applied in the integer finite field $\mathbb{Z}/p\mathbb{Z}$ (then it is named NTT) and more generally in the Galois fields $GF(p^n)$ [6, 7], and even in the quadratic fields [8, 9].

4 Roots of unity

The primitive N^{th} root of unity presented in the formulas of the DFT is the element w of the domain where

$$\begin{cases} w^N & = 1 \\ w^k & \neq 1 \quad \text{for all } k \text{ between 1 and } N-1 \end{cases} \tag{14}$$

We assume that $N > 1$ so such a w cannot exist in $\mathbb{R}$. Roots of unity are either complex numbers or belong to a finite field. In those two fields, there are some common interesting properties:

- When N is even, we have $w^{N/2} = -1$
- Since $w^N = 1$, it is clear that all exponents of w can be reduced modulo N. Taking the example of $N = 8$, the matrix (12) can be simplified by reducing all exponents modulo 8 (see section 5 for clarification on modular arithmetic):

$$T_8 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 0 & 2 & 4 & 6 & 0 & 2 & 4 & 6 \\ 0 & 3 & 6 & 1 & 4 & 7 & 2 & 5 \\ 0 & 4 & 0 & 4 & 0 & 4 & 0 & 4 \\ 0 & 5 & 2 & 7 & 4 & 1 & 6 & 3 \\ 0 & 6 & 4 & 2 & 0 & 6 & 4 & 2 \\ 0 & 7 & 6 & 5 & 4 & 3 & 2 & 1 \end{bmatrix} \tag{15}$$

When computing the DFT with a fast algorithm, it often makes sense to precalculate the roots of unity. In that case, only $N-1$ different exponents need to be evaluated, and not $(N-1)^2$ as we would have expected initially from the Definition 1 of the DFT.

4.1 in Complex-field

First let's explore complex roots of unity. Usually the authors start presenting the roots of unity in $\mathbb{C}$ with a formula difficult to understand for non-math persons. Let's take here another approach for didactic purposes. Let h be a function $\mathbb{R} \Rightarrow \mathbb{C}$ defined by

$$h(x) = cos(x) + isin(x) \tag{16}$$

Where i is the imaginary unit number ($i^2 = -1$).

Now let's evaluate this function for different values of x of the form $\frac{2\pi}{N}$ (where N varies). Observe the results below and deduct the logical sequence:

$$h(\frac{2\pi}{\mathbf{1}}) = 1 \qquad \text{and } 1^{\mathbf{1}} = 1$$
$$h(\frac{2\pi}{\mathbf{2}}) = -1 \qquad \text{and } (-1)^{\mathbf{2}} = 1$$
$$h(\frac{2\pi}{\mathbf{3}}) = -\frac{1}{2} + i\frac{\sqrt{3}}{2} \qquad \text{and } (-\frac{1}{2} + i\frac{\sqrt{3}}{2})^{\mathbf{3}} = 1$$
$$h(\frac{2\pi}{\mathbf{4}}) = i \qquad \text{and } i^{\mathbf{4}} = 1$$
$$h(\frac{2\pi}{\mathbf{8}}) = \frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2} \qquad \text{and } (\frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2})^{\mathbf{8}} = 1$$

We can observe that $h(\frac{2\pi}{N})$ raised to the power of N is equal to 1.
Meaning that the primitive N^{th} root of unity in $\mathbb{C}$ is

$$w = cos(\frac{2\pi}{N}) + isin(\frac{2\pi}{N}) \tag{17}$$

The roots of unity all lie on the circle of radius 1 unit of the complex plane. The different powers of w can be calculated using the following formula:

$$w^k = cos(\frac{2k\pi}{N}) + isin(\frac{2k\pi}{N}) \tag{18}$$

4.2 Example in Complex-field

Please note that the calculations below illustrate the formulas but are not intended to be fast. Remember that the DFT is not a fast algorithm.
As an example we can take again the initial polynomial function of the first section and proceed to the square of $f(x)$. This time $f(x)$ is a polynomial function in the complex field. For convenience, we will use $N = 8$ instead of $N = 7$. The reason is that the formula (18) is easier with $N = 8$ than with $N = 7$. Also, we will see later that fast optimizations are easier with power of two lengths. With $N = 8$, the input sequence is $A = \{1, 1, 1, 1, 0, 0, 0, 0\}$. So we have $a_0 = a_1 = a_2 = a_3 = 1$ and $a_4 = a_5 = a_6 = a_7 = 0$.

4.2.1 First step of the example: forward DFT

For the first step we need to calculate the 8th roots of unity using the formula (18). We get:

$$w = \frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2} \qquad w^2 = i \qquad w^3 = -\frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2} \qquad w^4 = -1$$
$$w^5 = -\frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2} \qquad w^6 = -i \qquad w^7 = \frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2} \qquad w^8 = 1$$

Now we proceed with the forward DFT using the formulas of definition 1:

$$\begin{aligned}
\widehat{a}_0 &= a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 \\
&= 1 + 1 + 1 + 1 \\
&= 4 \\
\widehat{a}_1 &= a_0 + a_1 w + a_2 w^2 + a_3 w^3 + a_4 w^4 + a_5 w^5 + a_6 w^6 + a_7 w^7 \\
&= 1 + w + w^2 + w^3 \\
&= 1 + (\frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2}) + i + (-\frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2}) \\
&= 1 + i(1 + \sqrt{2})
\end{aligned}$$

We continue using the same logic and reducing the powers of w modulo 8 ($w^9 = w$, $w^{10} = w^2$, ...)

$$\begin{aligned}
\widehat{a}_2 &= 1 + w^2 + w^4 + w^6 & \widehat{a}_3 &= 1 + w^3 + w^6 + w \\
&= 0 & &= 1 + i(-1 + \sqrt{2}) \\
\widehat{a}_4 &= 1 + w^4 + 1 + w^4 & \widehat{a}_5 &= 1 + w^5 + w^2 + w^7 \\
&= 0 & &= 1 + i(1 - \sqrt{2}) \\
\widehat{a}_6 &= 1 + w^6 + w^4 + w^2 & \widehat{a}_7 &= 1 + w^7 + w^6 + w^5 \\
&= 0 & &= 1 + i(-1 - \sqrt{2})
\end{aligned}$$

4.2.2 Second step of the example: square

Now that we have $\widehat{A}$ let us calculate $\widehat{B} = \widehat{A} \times \widehat{A}$ by a pointwise-square: $\widehat{b}_k = \widehat{a}_k \times \widehat{a}_k$

$$\begin{aligned}
\widehat{b}_0 &= 16 & \widehat{b}_1 &= (-2 - 2\sqrt{2}) + i(\ \ 2 + 2\sqrt{2}) \\
\widehat{b}_2 &= 0 & \widehat{b}_3 &= (-2 + 2\sqrt{2}) + i(-2 + 2\sqrt{2}) \\
\widehat{b}_4 &= 0 & \widehat{b}_5 &= (-2 + 2\sqrt{2}) + i(\ \ 2 - 2\sqrt{2}) \\
\widehat{b}_6 &= 0 & \widehat{b}_7 &= (-2 - 2\sqrt{2}) + i(-2 - 2\sqrt{2})
\end{aligned}$$

4.2.3 Third step of the example: Inverse DFT

At this stage we need the inverse roots of unity. We just need to invert to sign of the imaginary part of the roots of unity:

$$w^{-1} = \frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2} \qquad w^{-2} = -i \qquad w^{-3} = -\frac{\sqrt{2}}{2} - i\frac{\sqrt{2}}{2} \qquad w^{-4} = -1$$
$$w^{-5} = -\frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2} \qquad w^{-6} = i \qquad w^{-7} = \frac{\sqrt{2}}{2} + i\frac{\sqrt{2}}{2} \qquad w^{-8} = 1$$

We proceed with the inverse DFT using the formulas of definition 2:

$$\begin{aligned} b_0 &= \frac{1}{8}(\widehat{b}_0 + \widehat{b}_1 + \widehat{b}_2 + \widehat{b}_3 + \widehat{b}_4 + \widehat{b}_5 + \widehat{b}_6 + \widehat{b}_7) \\ &= \frac{1}{8}(16 - 2 - 2 - 2 - 2) \\ &= 1 \end{aligned}$$

During the calculation of b_0 we observe that the imaginary part disappears, and so do the irrational numbers. This kind of magic happens again with the subsequent elements despite more painful calculations:

$$\begin{aligned} b_1 &= \frac{1}{8}(\widehat{b}_0 + \widehat{b}_1 w^{-1} + \widehat{b}_2 w^{-2} + \widehat{b}_3 w^{-3} + \widehat{b}_4 w^{-4} + \widehat{b}_5 w^{-5} + \widehat{b}_6 w^{-6} + \widehat{b}_7 w^{-7}) \\ &= \frac{1}{8}(16 + i(2\sqrt{2} + 4) + 0 + i(2\sqrt{2} - 4) + 0 + i(-2\sqrt{2} + 4) + 0 + i(-2\sqrt{2} - 4)) \\ &= \frac{1}{8}(16) \\ &= 2 \end{aligned}$$

The last steps of calculations are omitted here but with courage we get the final results:

$$B = \{1, 2, 3, 4, 3, 2, 1, 0\}$$

which is the expected result, see (3).

4.3 Another notation

We can rewrite differently the roots of unity using Euler's formula which states that for any real number x:

$$e^{ix} = cos(x) + isin(x) \qquad \text{(Euler's formula)} \tag{19}$$

Using this notation, we have

$$w = e^{i\frac{2\pi}{N}} \tag{20}$$

$$w^k = e^{i\frac{2k\pi}{N}} \tag{21}$$

This notation is more compact and is much more used than (17) and (18). However, for persons not having a superior math knowledge, we must admit it is obscure. At school, the concept of exponentiation is explained in different steps. First with positive integer exponents: the kids understand well that this is just a repeated multiplication. Then negative integer exponents are introduced with fractions. And later they learn that the exponent can be fractional or even any real number. But we don't learn at school that the exponent can be a complex number. Indeed, how can we raise a number to the power of i ? The concept of complex exponentiation is harder to visualize than a repeated multiplication.

Another source of misunderstanding for students comes from the fact the DFT is often defined as follow:

$$\widehat{a}_k = \sum_{j=0}^{N-1} a_j e^{\frac{2ijk\pi}{N}} \tag{22}$$

This is not the definition of the DFT as it reduces the transform to the complex-field. Introducing the DFT with such a definition does not help the students to understand the big picture.

In addition presenting ijk close together in this equation brings confusion. For any IT person, `i j k` are commonly indices in a loop. Here that's the case for j and k but not for i which represents the imaginary unit number. These three sources of confusion (raising a number to the power of i, introducing the DFT in the Complex-field, and presenting ijk together) clearly explain the confusion of students and beginners who considered these topics difficult to understand.

It also has to be said that the term *Fast Fourier Transform* or FFT is often used in lieu of the DFT. Then the students do not clearly understand that the *Fast Fourier Transform* is not a *Transform* but an *algorithm* to compute the DFT efficiently. That explains why the FFT has not been introduced yet in this document.

That being said, and for a recreational conclusion to this section dedicated to complex numbers, we should cite *Euler's identity*. This identity is a special case of the Euler's formula (19) for $x = \pi$:

$$e^{i\pi} + 1 = 0 \qquad \text{(Euler's identity)} \tag{23}$$

Euler's identity is often cited simply as the most beautiful equation ever written. This is because it links the five most fundamental constants (0, 1, π, i and e) in an elegant and short manner, and with the simplest arithmetic operations (addition, multiplication and exponentiation). Then it is a bridge between different areas of mathematics (algebra, geometry and trigonometry, analysis).

This is closing the section about complex numbers. We will now focus exclusively on integers.

5 Modular Arithmetic

We will now work in the finite field of integers $\mathbb{Z}/p\mathbb{Z}$ where p is prime. Let us write $\mathbb{Z}_p$ as a synonym, it is shorter. Please note that the principles explained here will sometimes be extended to the ring $\mathbb{Z}_n$ (n not a prime) when explicitly mentioned.

What is $\mathbb{Z}_p$? For the sake of simplicity you can see $\mathbb{Z}_p$ as the set of integers from 0 to $p-1$ where all calculations are reduced modulo p. For example $\mathbb{Z}_7$ has 7 elements $\{0, 1, 2, 3, 4, 5, 6\}$ and all calculations are done modulo 7. In the literature you can also find the terms *set of residues* and *residue system*.

For example, $2 + 6 = 1$ in $\mathbb{Z}_7$. This may seem a strange way to do arithmetic but we do modular arithmetic everyday with hours (mod 12 and mod 24) and days of the week (mod 7): 2 days after the 6th day of the week is the 1st day of the week. This is noted usually as $2 + 6 \equiv 1 \pmod 7$ or simply $2 + 6 = 1 \mod 7$. We will use the later notation in this document. To complete this overview over the naming conventions, we also say that 8 and 1 are congruent modulo 7, and that 1 is the remainder of the integer division of 8 by 7.

Many programming languages have a `Mod()` function or even an operator usually defined as `%`. For example `Mod(8, 7)` yields 1, and also `8 % 7`. However, an efficient DFT in $\mathbb{Z}_p$ should not use these functions which are generally slow. This may be hard to achieve regarding the definition of the NTT in the next section but different tricks can be applied.

- First NTT (with or without FFT algorithms) perform a lot of repeated arithmetic operations. It is faster to keep each intermediate result within the range $[0, p-1]$ by testing the overflow instead of doing all the arithmetic operations then reducing a big total modulo p.
- Modulo can be translated into the bitwise operation `AND` if the modulus is a power of two, so having the form 2^k. In that case, performing a bitwise `AND` $(2^k - 1)$ gives the same result and is faster. For example consider the following code:

  ```
  int value = a % 256;
  ```

  It can be efficiently replaced by:

  ```
  int value = a & 255;
  ```
- More generally, if the modulus has a special form like $2^j \pm 2^k \pm 1$, it can be translated into bitwise operations.

The DFT can be calculated in $\mathbb{Z}_p$ and it is then called a Number Theoretic Transform (NTT).

6 NTT

6.1 Definition

Definition 3. Number Theoretic Transform (NTT) in $\mathbb{Z}_p$ (p is prime)
Let A be a sequence of N integers a_j all in the range from 0 to $p-1$.

$$A = \{a_0, a_1, ..., a_{N-1}\}$$

The NTT of A is the transformed sequence $\widehat{A} = \text{NTT}(A)$ defined by the following N elements, all integers

$$\widehat{a}_k = \sum_{j=0}^{N-1} a_j w^{jk} \mod p \tag{24}$$

The inverse transform $A = \text{INTT}(\widehat{A})$ is defined by the following N elements, all integers

$$a_j = N^{-1} \sum_{k=0}^{N-1} \widehat{a}_k w^{-jk} \mod p \tag{25}$$

where w is the primitive N^{th} root of unity modulo p, meaning that

$$\begin{cases} w^N & = 1 \mod p \\ w^k & \neq 1 \mod p \quad \text{for all } k \text{ between 1 and } N-1 \end{cases} \tag{26}$$

and

$$NN^{-1} = 1 \mod p \tag{27}$$

6.2 Example in $\mathbb{Z}_p$

To illustrate the NTT we take again the example of the first section of this document, by squaring $f(x) = 1+x+x^2+x^3$. We choose $p = 257$ and will discuss later this choice.
After observing that

$$\begin{aligned} 2^8 = 256 &= -1 \mod 257 \\ 4^8 = 65536 &= \ \ 1 \mod 257 \end{aligned}$$

we can check that 4 is the primitive 8th root of unity in $\mathbb{Z}_{257}$ which is perfect because we wanted to keep $N = 8$ by symmetry with the previous example in the Complex-field. So the three variables of the NTT are:

$$p = 257 \qquad N = 8 \qquad w = 4$$

The input sequence is built from the coefficients of $f(x)$ just as in the example in the Complex-field: $A = \{1, 1, 1, 1, 0, 0, 0, 0\}$ so we have $a_0 = a_1 = a_2 = a_3 = 1$ and $a_4 = a_5 = a_6 = a_7 = 0$.

6.2.1 First step of the example: forward DFT

First the different roots of unity are calculated and reduced modulo 257:

$$w = 4 \qquad w^2 = 16 \qquad w^3 = 64 \qquad w^4 = 256 \qquad w^5 = 253 \qquad w^6 = 241 \qquad w^7 = 193 \qquad w^8 = 1$$

Then we can proceed with the forward transform:

$$\begin{aligned} \widehat{a}_0 &= a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 \\ &= 1 + 1 + 1 + 1 \\ &= 4 \\ \widehat{a}_1 &= a_0 + a_1 w + a_2 w^2 + a_3 w^3 + a_4 w^4 + a_5 w^5 + a_6 w^6 + a_7 w^7 \\ &= 1 + w + w^2 + w^3 \\ &= 1 + 4 + 16 + 64 \\ &= 85 \end{aligned}$$

Using the same logic we calculate the subsequent elements reduced modulo 257:

$$\begin{aligned}\widehat{a}_2 &= 1 + w^2 + w^4 + w^6 \\ &= 0 \\ \widehat{a}_4 &= 1 + w^4 + 1 + w^4 \\ &= 0 \\ \widehat{a}_6 &= 1 + w^6 + w^4 + w^2 \\ &= 0\end{aligned} \qquad \begin{aligned}\widehat{a}_3 &= 1 + w^3 + w^6 + w \\ &= 53 \\ \widehat{a}_5 &= 1 + w^5 + w^2 + w^7 \\ &= 206 \\ \widehat{a}_7 &= 1 + w^7 + w^6 + w^5 \\ &= 174\end{aligned}$$

So the transformed result is

$$\widehat{A} = \{4, 85, 0, 53, 0, 206, 0, 174\}$$

6.2.2 Second step of the example: square

Then we calculate $\widehat{B} = \widehat{A} \times \widehat{A}$ by a pointwise-square: $\widehat{b}_k = \widehat{a}_k \times \widehat{a}_k \mod 257$

$$\widehat{B} = \{16, 29, 0, 239, 0, 31, 0, 207\}$$

6.2.3 Third step of the example: Inverse DFT

At this stage we need the inverse roots of unity. These are easily deducted from the standard roots of unity because:

$$w \times w^7 = w^8 = 1$$

So w^7 is the inverse of w. More generally $w^{-k} = w^{N-k}$. So we get the inverse roots of unity by taking the already calculated roots of unity in the reverse order:

$w^{-1} = 193 \qquad w^{-2} = 241 \qquad w^{-3} = 253 \qquad w^{-4} = 256 \qquad w^{-5} = 64 \qquad w^{-6} = 16 \qquad w^{-7} = 4 \qquad w^{-8} = 1$

For the inverse DFT, we also need N^{-1} the inverse of 8. We can observe that

$$\begin{aligned}257 &= 32 \times 8 + 1 \\ -32 \times 8 &= 1 - 257 \\ -32 \times 8 &= 1 \mod 257\end{aligned}$$

Meaning that -32 is the inverse of 8. And -32 is equal to 225 modulo 257. Indeed we can check that $225 \times 4 = 1 \mod 257$.

$$N^{-1} = 225$$

Now we can proceed with the calculation of the inverse DFT:

$$\begin{aligned}b_0 &= N^{-1}(\widehat{b}_0 + \widehat{b}_1 + \widehat{b}_2 + \widehat{b}_3 + \widehat{b}_4 + \widehat{b}_5 + \widehat{b}_6 + \widehat{b}_7) \\ &= 225(16 + 29 + 0 + 239 + 0 + 31 + 0 + 207) \\ &= 225 \times 8 \\ &= 1 \\ b_1 &= N^{-1}(\widehat{b}_0 + \widehat{b}_1 w^{-1} + \widehat{b}_2 w^{-2} + \widehat{b}_3 w^{-3} + \widehat{b}_4 w^{-4} + \widehat{b}_5 w^{-5} + \widehat{b}_6 w^{-6} + \widehat{b}_7 w^{-7}) \\ &= 225(16 + (29 \times 193) + 0 + (239 \times 253) + 0 + (31 \times 64) + 0 + (207 \times 4)) \\ &= 225 \times 16 \\ &= 2\end{aligned}$$

The subsequent elements are not more difficult to evaluate, and we get the final results:

$$B = \{1, 2, 3, 4, 3, 2, 1, 0\}$$

which is the expected result, see (3).

6.3 Finding the inverse in $\mathbb{Z}_p$

In the previous example we have deducted the value of N^{-1} by observation. This section explains how to automate this calculation and find the inverse a^{-1} so that $aa^{-1} = 1 \mod p$. Such an inverse always exists in $\mathbb{Z}_p$ because p is prime. When p is composite, the existence of the inverse can sometimes exist under some circumstances (ref needed) thus allowing the NTT to be performed. But as already mentionned we focus here on the way to find the inverse of a in the field $\mathbb{Z}_p$ where p is prime.
The extended Euclidean algorithm is a good way to find x and y such as $ax + py = \gcd(a, p) = 1$. If you have $ax + py = 1$ it means that x is the inverse of a because the part py disappears modulo p.

But instead of the Euclidean algorithm, the Fermat's little theorem can be a better choice because of its simplicity from a computer coding perspective.

$$a^p = p \mod p \qquad \text{(Fermat's Little Theorem)} \tag{28}$$

$$a^{p-1} = 1 \mod p \qquad \text{because } a \text{ is not divisible by } p$$

$$aa^{p-2} = 1 \mod p \tag{29}$$

Meaning that a^{p-2} is the inverse of a. So N^{-1} can be replaced by N^{p-2} in the definition of the inverse NTT. You just have to repeatedly multiply N and reduce each intermediate result modulo p in order to avoid overflow if p is large. This is clearly not the fastest algorithm but acceptable for the NTT because it is done in a precalculation phase.

6.4 Finding the primitive N^{th} root of unity modulo p

First it has to be said that the *primitive N^{th} roots of unity modulo p* should not be confused with the *primitive roots modulo p*. Just for information here are some explanations on the later (skip this paragraph if not interested). The *primitive root modulo p* is a special case of the *primitive N^{th} roots modulo p* where $N = \varphi(p)$, φ being the Euler's totient function. This function is always equal to $p - 1$ in the case where p is prime. In that case the *primitive root modulo p* is precisely the *primitive $(p-1)^{th}$ root modulo p*. Those primitive roots modulo p are convenient because they are a generator in the sense that all powers from 1 to $p - 1$ give all the distinct residue values of $\mathbb{Z}_p$. When p is prime such a primitive $(p-1)^{\text{th}}$ root of unity exists. This concludes this short digression.

Let W be a primitive $(p-1)^{\text{th}}$ root of unity modulo p. According to definition 14 we have:

$$\begin{cases} W^{p-1} & = 1 \mod p \\ W^k & \neq 1 \mod p \quad \text{for all } k \text{ between 1 and } p-2 \end{cases} \tag{30}$$

Such W exists (p is prime) and we could call that the *deepest* primitive N^{th} root of unity. Meaning that $p - 1$ is the largest possible NTT length. How can we find such a W ? In the previous example we found quite easily that 4 is a 8th primitive root of unity modulo 257. But it may not be so obvious to guess that 3 is a 256th primitive root of unity modulo 257 (you can check that $3^{256} = 1 \mod 257$ and that all powers of 3 from 1 to 256 give distinct numbers in the range from 1 to 256). Indeed not easy to guess, and unfortunately there is no formula to compute W. Let us define an algorithm to find W.

We first note that $W^{p-1} = 1 \mod p$ is necessary true for every W according to the Fermat's little theorem, see (28). Thus we focus on the second condition, and we take a brute force approach by testing each integer one by one starting from 2. We first test if 2 is the primitive root we are searching for. If this is not the case, we check 3, then we can skip 4 because 4 is a perfect square and it cannot be the primitive root we are searching for, then 5, etc.
Checking each of these candidates is not so difficult because there exists a faster method than evaluating $W^k \mod p$ for each k between 1 to $p - 2$. In fact we can only test the prime factors of $p - 1$. If one of them returns 1 then this is not the primitive root we are searching for. If for all of them the result of the exponentiation is different from 1 then we get our W.

Algorithm 1. Returns the primitive $(p-1)^{\text{th}}$ root of unity modulo p
Assuming that the prime factors of $p - 1$ are stored in an array `PF`
And that `pow_Mod()` is a function that computes the exponentiation modulo p

```
function GetDeepestRoot(p) {
  var W = 1;
  var Found = false;
```

```
  while (!Found) {
    Found = true;
    W++;
    foreach (var prime in PF) {
      if (pow_Mod(W, prime) == 1) {
        Found = false;
        break;
      }
    }
  }
  return W;
}
```

The Algorithm 1 calculates W and from that W we can easily deduct w the primitive N^{th} root of unity, for any N that divides $p-1$.

$$w = W^{(p-1)/N} \tag{31}$$

This method has been successfully implemented in the forward section (ref needed) for the modulus $p = 2^{64} - 2^{32} + 1$. The algorithm has been slightly improved to skip perfect squares, please see github.

As a conclusion to this section we remark that there is a great advantage of having $p-1$ highly composite when calculating a NTT. This is because we can choose the best prime factors of $p-1$ (ideally high powers of 2) to compose the length N, and it leads to FFT algorithms.

6.5 Choosing the perfect NTT

NTT have a major advantage over Complex Transforms: all calculations are done using integers, and it is free of any round-off error due to floating point calculations. When calculating on a computer a complex DFT (whether or not using a FFT method) you cannot avoid floating point imprecision. Today these round-off errors seem to be under control, but mostly through empirical guesses.
Let me cite Crandall:
- "*The problem of bounding convolution errors is extremely difficult*" [10]
- "*convolution errors are not completely understood*" [10]
- "*very little is known in the way of error bounds that are both rigorous and pragmatic*" (CP05 ref needed)
Moreover... In terms of elegance it is still difficult to admit that to perform a large integer multiplication, or to prove the primality of an integer, we need to use millions of times the `Round()` function (or similar) to convert a `double` to an `integer`.

NTT do not have this problem, the calculations are done using integers. However, it also has major drawbacks. According to the popular opinion, NTT are said to be slow (ref need CP03 page 6) and not suitable for very large transforms. This can be discussed but however what is undeniably true is the strong link between the choice of the modulus p, the possible length N and the ease of calculation. When you are developing a NTT you have the choice of p, N and w. So the number of possible NTT seems very large at first glance. Closer examination shows that there exists a strong link between those three variables which reduces the scope of possibilities. The choice of the modulus p drives everything:

- p should be large enough if you want to run large transforms. The maximum length of the DFT is restricted by p because the maximum possible length is $N = p - 1$. Moreover, the size of each element within the transform is also limited by p to avoid overflow.
- p should not be too large in order to do the calculations modulo p efficiently. A good idea is probably to keep p within a computer word, for example 32 or 64 bits.
- p should have a special form in order to calculate the modulo easily, without division. The modulo operation is a slow operation on computers, except if the modulus has a special form. We will discuss this topic later.
- Ideally roots of unity should also have a special form in order to do the multiplications easily (if possible with shifts).

- $p-1$ should be highly composite (if possible with powers of 2) as shown in the previous section, in order to enable FFT algorithms.

Let us evaluate the different possibilities in the historical order of their discovery. For additional bibliography references please read Creutzburg et al (CL91 and BCA04).

6.5.1 The early papers and the Chinese Remainder Theorem (CRT)

In 1971 Pollard (ref needed) and Shönhage/Strassen (ref needed) were among the first authors to mention the usage of the finite fields to compute the multiplication of very large integers. Pollard suggested to compute several transforms with some appropriate set of primes $\{p_i\}$ and to reconstruct the final result using the Chinese Remainder Theorem. Please read Crandall (Cra05 ref needed) for descriptive algorithms.
During decades NTT have been associated with the CRT in the popular opinion. This is due to the fact that the authors did not find appropriate sets of variables $\{p, N, w\}$ to compute large transforms efficiently. Thus the CRT appeared as a clear evidence to compute larger convolutions at the cost of an additional step (the reconstruction). Let us have a look on these attempts to find promising moduli, starting by Mersenne numbers.

6.5.2 Mersenne Number Transforms

Mersenne Number Transforms (MNT) are NTT with a modulus of the following form:

$$p = M_k = 2^k - 1 \qquad k \text{ being a prime} \tag{32}$$

These moduli are called Mersenne Numbers. Despite the fact that k is prime, very few Mersenne Numbers are prime. However Rader (Rad72, ref needed) has shown that MNT can be performed with composite Mersenne Numbers which extends the possible range of MNT.
At first glance these transforms may look very interesting because we can use the following set of variables:

$$p = 2^k - 1 \qquad N = k \qquad w = 2 \qquad k \text{ being a prime}$$

Having $w = 2$ is a great advantage because we can perform the NTT without multiplications, only with shifts. However N being a prime is not convenient at all for fast optimizations. Furthermore, the size N of the transform is far too small to be of some interest for practical use. Even if Rader has shown that the available length can be extended to $2k$, Mersenne number transforms are not of as much interest.

6.5.3 Fermat Number Transforms

In the previous example of the section 6.2 we used the modulus $p = 257$ with the following variables

$$p = 257 \qquad N = 8 \qquad w = 4$$

But a closer examination shows that we could have used those other choices:

$$p = 257 \qquad N = 16 \qquad w = 2$$
$$p = 257 \qquad N = 256 \qquad w = 3$$

This transform is called a Fermat Number Transform because 257 is a Fermat number ($2^{2^3} + 1$). More generally Fermat Number Transforms (FNT) are NTT with a modulus of the following form:

$$p = F_k = 2^{2^k} + 1 \tag{33}$$

These transforms have been discussed by Agarwal and Burrus (ref needed). We already illustrated the case $k = 3$ (which gives $p = 257$). Let us now discuss the subsequent possibilities. With $k = 4$ we have the different options (among others):

$$p = 65537 \qquad N = 32 \qquad w = 2 \tag{34}$$
$$p = 65537 \qquad N = 64 \qquad w = \sqrt{2} = 4080 \tag{35}$$
$$p = 65537 \qquad N = 65536 \qquad w = 3 \tag{36}$$

The first option (34) looks very promising because we can perform the transform only with additions and shifts instead of multiplications (this arithmetic being done within 32-bit registers), and N is a power of two which opens the door to fast algorithms with FFT. This can potentially leads to the fastest multiplication algorithm. However this is reserved for

tiny transforms. Even with options (35) and (36) which are extending the length N of the transform, only numbers with a few hundreds of bits can be multiplied without overflow.
The subsequent Fermat numbers seem to be all composite, but they still can be used to compute the FNT as demonstrated by Agarwal and Burrus. With $k = 5$ and $k = 6$ you are still limited to small transforms. And if you increase k you rapidly get an unusable modulus.
To conclude with FNT we should also mention that Nussbaumer (nussbaumer1977 in SAS, ref needed)) has investigated pseudo Fermat numbers of the form $2^k + 1$ where k is highly composite but not necessary a power of two.

6.5.4 Moduli of the form $2^j - 2^k + 1$

After the papers about the Fermat Number Transform and their limitations, Pollard (POL76 ref needed) has considered the use of moduli of the form

$$p = 2^j - 2^k + 1 \qquad \text{where } j > k$$

in order to circumvent the disadvantages of the FNT, but without giving good practical examples.
Then Liu et al (LRT76 ref needed) has suggested the form

$$p = 2^{2k} - 2^k + 1$$

which is a special case of the form considered by Pollard. They investigated different values of k and mentioned that the first prime numbers to be of this form is the interesting

$$p = 2^{64} - 2^{32} + 1$$

They were the first to remark that this prime enables NTT with a maximum length of 2^{32}.
Dubois et al (DV78 ref needed) have extended their work and considered moduli of the form

$$p = (2^{2q} - 2^q + 1)/k$$

Several moduli have been presented, and again $p = 2^{64} - 2^{32} + 1$ seems to be the best choice regarding the maximum possible length, the efficiency of wordlength use, and the ease of calculations.

6.5.5 Other moduli

Colomb et al (Golomb77 in SAS, ref needed) have considered moduli of this form:

$$p = 3 \times 2^k + 1$$

The same year, Reed et al (reed1977 in SAS, ref needed) has extended their work with the more general form:

$$p = q \times 2^k + 1$$

In particular, the prime $p = 45 \times 2^{29} + 1$ has been chosen for illustrating their purpose. More recently, Papadopoulos (Crai02, ref needed) has suggested other moduli of the same form, but much bigger.

6.6 The modulus $p = 2^{64} - 2^{32} + 1$

After these more or less successful investigations in the seventies, NTT have slowly lost their popularity probably at the same rate than the need for larger and faster transforms. However it is the author's conviction that an error-free full-integer multiplication implementation should exist for at least two reasons: undoubtedly exactness and elegance.

With these objectives, the following modulus cited by Liu and Dubois looks the most promising and should be investigated:

$$p = 2^{64} - 2^{32} + 1 \tag{37}$$

A successful implementation of this modulus was coded in the 2000's by Craig-Wood (ref needed) on the ARM processor. This work has clearly inspired me and I am very grateful to David Craig-Wood.

LucasNTT (ref needed) is based on this modulus. The second part of this document explains step by step how it has been implemented.

7 Convolutions

In the very first section of this document we saw that a simple transform can simplify the multiplication of polynomials. The objective was not to speed up the process but to show how transforms can modify an algorithm by changing the point of view. Then two examples in section 4.2 and section 6.2 show how to achieve the same result with the DFT and the NTT. This section explains the underlying mechanism and will lead to the DFT multiplication algorithm for large integers. But first we need to explain the term *convolution* because it is the core component.

7.1 Acyclic convolutions

Recall the school multiplication (term tbc) and let's try to do such a multiplication without performing any carry. Let's say we multiply 1234 by 5678 without carrying. The example below is inspired from Wikipedia (ref needed).

$$\begin{array}{cccccccc}
 & & & 1 & 2 & 3 & 4 \\
 & & \times & 5 & 6 & 7 & 8 \\
\hline
 & & & 8 & 16 & 24 & 32 \\
 & & 7 & 14 & 21 & 28 & \\
 & 6 & 12 & 18 & 24 & & \\
5 & 10 & 15 & 20 & & & \\
\hline
5 & 16 & 34 & 60 & 61 & 52 & 32
\end{array} \tag{38}$$

The resulting sequence is called the "acyclic convolution" of the two initial sequences. If the initial sequences represent the coefficients of two polynomials, then the acyclic convolution represents the coefficients of their ordinary product. This is known as the Cauchy product. We can check that:

$$(x^3 + 2x^2 + 3x + 4)(5x^3 + 6x^2 + 7x + 8) = 5x^6 + 16x^5 + 34x^4 + 60x^3 + 61x^2 + 52x + 32 \tag{39}$$

This is to say that the acyclic convolution of the two sequences $\{4, 3, 2, 1\}$ and $\{8, 7, 6, 5\}$ is the resulting sequence $\{32, 52, 61, 60, 34, 16, 5\}$. Please note that by convention chosen at the beginning of this document, we start by the lowest order when writing sequences into curly brackets.
In order to write a formal definition of the acyclic convolution in the math language, observe how each element of the schoolbook multiplication has been calculated. For example observe that 61 is the sum of three products of two initial elements $(8 \times 2) + (7 \times 3) + (6 \times 4)$, and that these initial elements have been grouped by pair $(8, 2)$, $(7, 3)$ and $(6, 4)$ because in each pair the sum of the indices is equal to the indice of the column. This was a long sentence that can be simply written in the math language: $c_k = \sum_{i+j=k} a_i \times b_j$

Definition 4. Acyclic convolution
Let A and B be two sequences made of N elements a_i and b_j.

$$A = \{a_0, a_1, ..., a_{N-1}\}$$
$$B = \{b_0, b_1, ..., b_{N-1}\}$$

The acyclic convolution is the resulting sequence C made of $2N - 1$ elements c_k defined by

$$c_k = \sum_{i+j=k} a_i \times b_j \qquad k \text{ between } 0 \text{ and } 2N - 2 \tag{40}$$

The acyclic convolution is also called "linear convolution" and is usually written $A * B$.

We assume here that both initial sequences have the same length. If not, zero pad the shortest one to extend its length.

7.2 Acyclic convolutions

Now imagine that we don't have enough space in the resulting sequence because it is limited to N elements. The school multiplication (term tbc) may become:

$$\begin{array}{ccccc}
 & 1 & 2 & 3 & 4 \\
\times & 5 & 6 & 7 & 8 \\
\hline
 & 8 & 16 & 24 & 32 \\
 & 14 & 21 & 28 & {}^\star 7 \\
 & 18 & 24 & {}^\star 6 & {}^\star 12 \\
 & 20 & {}^\star 5 & {}^\star 10 & {}^\star 15 \\
\hline
 & 60 & 66 & 68 & 66
\end{array} \tag{41}$$

As an image we can say that the items with the star have been sent to the right in a circular way because of the lack of available space on the left.
The resulting sequence is called the "cyclic convolution" of the two initial sequences.

Definition 5. Cyclic convolution
Let A and B be two sequences made of N elements a_i and b_j.

$$A = \{a_0, a_1, ..., a_{N-1}\}$$
$$B = \{b_0, b_1, ..., b_{N-1}\}$$

The cyclic convolution is the resulting sequence C made of N elements c_k defined by

$$c_k = \sum_{i+j=k \mod N} a_i \times b_j \qquad k \text{ between } 0 \text{ and } N-1 \tag{42}$$

The cyclic convolution is also called "circular convolution" and is usually written $A \circledast B$.

Why are the convolutions so important for the multiplication algorithm?
Because the multiplication of large integers can be computed from an acyclic convolution.
And the acyclic convolution can be computed from a cyclic convolution.
And the cyclic convolution can be computed from a DFT or a NTT.
And the DFT and the NTT can be computed with fast algorithms.

Let us take those affirmations one by one. Once we have the acyclic convolution of two sequences we can compute the product of the two initial numbers quite easily by adjusting the carry in the chosen base. The carry propagation is made from the rightmost element to the left in one single pass. In the above example (1234×5678), we are working with the decimal system (base 10) which is not the most convenient from a computational perspective but clearly the most appropriate here for the demonstration. The rightmost element is 32, we keep 2 and we add 3 to the left neighbour, which becomes 55, we keep 5 and we add 5 to the left neighbour, and so on. The final result yields $7'006'652$ which is the expected product.

The problem is now reduced to the computation of the acyclic convolution. It can be computed from the cyclic convolution by different means, but the most obvious one is to allocate enough space during the computation of the cyclic convolution. By allocating $2N$ elements for the cyclic convolution instead of N, the cyclic folding will not occur because we are canceling the modulo N effect that appears in the definition 5. So the idea is to zero pad the two input sequences to a $2N$ length.

The problem is now reduced to the computation of the cyclic convolution. It can be computed with the DFT thanks to the *Convolution theorem*.

7.3 Convolution theorem

The Convolution theorem states that some transforms naturally perform a cyclic convolution when computing the pointwise product of their transformed sequences. We say that these transforms have the convolution property. And it can be shown (ref needed) that Discrete Fourier Transforms (including pure integer NTT) have this property.

The cyclic convolution $A \circledast B$ is computed by the following steps: transform the two input sequences A and B, then multiply each single element (for each j calculate $a_j \times b_j$), then transform back the result to the original domain (inverse transform).

$$A \circledast B = \text{IDFT}(\text{DFT}(A).\text{DFT}(B)) \tag{43}$$

As a conclusion to this *Convolution* section we can say that the complexity of the DFT multiplication algorithm for large integers is mainly reduced to the complexity of the steps that involve the Discrete Fourier Transform.

8 Multiplication Algorithm for large integers

In the previous section we started to design a draft of the multiplication algorithm as follow: zero pad the input sequences, then perform a DFT (or NTT), then compute a pointwise product, then perform a backward IDFT (or INTT), then adjust the carry in the chosen base.
But there are some initial steps in order to compute the length of the DFT and to prepare the input numbers.

8.1 Evaluating the length of the DFT

Basically, the size of your input numbers will determine the minimum length of the DFT. The DFT operates with a sequence made of elements. The size of these elements is constrained by precision limits. DFT with complex numbers commonly use double-precision floating-point format (64 bits) but need to dramatically reduce the number of bits per element in order to keep the round-off errors under control (ref needed).

For NTT, the size of each element is first constrained by the modulus p. But it is also dramatically reduced to avoid overflow during calculation. (explain the formula that leads to NX^2)

Let's take an example but for the sake of simplicity we choose to square (prendre un exemple réel de test LL avec un Mersenne prime) The game is to choose the shortest DFT size, but also the most convenient one regarding the fast optimizations (possibly with powers of two) and which can contain twice the elements of your initial large integers without overflow.

Imagine you want to multiply an integer A which has 10'000'000 bits (A is around $2^{10000000}$ by another smaller integer B. We use a NTT with our favorite modulus $p = 2^{64} - 2^{32} + 1$.

The next section will explain these constraints. For now let's say that you have evaluated that the size of each element should not exceed 24 bits. Then the large integers need to be cut into 24-bit chunks. The size of the largest integer will determine the number of chunks. Then you need to double this number to get an acyclic convolution, and from this number you will determine the size N of the DFT.

8.2 Initiating the DFT multiplication

However there may be an initial step which is to represent the input parameters in a sequence compatible with the DFT. Depending on your context you may receive input parameters (the two large integers you want to multiply) that need to be cut into small pieces.

If you are not exposing your multiplication function to other clients i.e. if you are the owner of the code that uses this function, you may decide to store large integers differently. This is particularly the case during a Lucas-Lehmer test (ref needed) where repeating squares are necessary. In that case it seems more efficient to continuously keep the large integer in an appropriate base representation that does not overflow. Then the step (1) can be skipped.

8.3 The DFT multiplication algorithm

Algorithm 2. DFT multiplication $C = A \times B$
(1) Compute N the length of the DFT following previous algorithm
(2) Put the initial numbers in base β compatible with N and zero pad following previous algorithm
(3) $\widehat{A} = DFT(A)$ // possibly with FFT algorithm
$\widehat{B} = DFT(B)$ // possibly with FFT algorithm
(4) Pointwise product
$c_k = \widehat{a}_k \times \widehat{b}_k$
(5) $C = IDFT(\widehat{C})$
(6) Reduce to base

9 Multiplication Algorithm modulo q

10 IBDWT

11 FFT

Taxonomy: radix algorithms vs other algorithms, the 4 core algorithms, New way of presenting using graphs strides New algorithm of Pease (myFFT) New algorithm with permutations

Part II
LucasNTT Implementation

12 Arithmetics

Let us now see how we can calculate the DFT using this modulus. First it has to be said that we need 64 bits in order to store an element of $\mathbb{Z}_p$. It is quite convenient because it fits exactly into a long word for most of the computer. By using 64 bits words we do not loose any bit thus increasing the transform efficiency. Liu (?) has calculated the efficiency at 99.99% (ref needed)

12.0.1 Arithmetic in this field

We need to investigate how to do arithmetic in this field. Please note that the programming codelets in the following section are in C and are the very first lines of code of LucasNTT. We start by the subtraction which is the easiest operation:

```
#include <inttypes.h>

const uint64_t p = 0xFFFFFFFF00000001; // Our modulus p = 2^64 - 2^32 + 1

uint64_t sub_Mod(uint64_t x, uint64_t y)
{
        uint64_t result = x - y;
        if (x < y)
                result += p;
        return result;
}
```

During the addition of `x + y` we should be careful to keep the result within the range `[0..p-1]` and also to avoid overflow below 2^{64}. Instead of performing two tests Craig-Wood suggested to calculate `y - (p - x)`.

```
uint64_t add_Mod(uint64_t x, uint64_t y)
{
        x = p - x;
        uint64_t result = y - x;
        if (y < x)
                result += p;
        return result;
}
```

The multiplication of two elements in $\mathbb{Z}_p$ yields a 128-bit result which needs to be reduced modulo p. We want to avoid the `C++` modulo operator which is slow. We take advantage of the following observations:

$$2^{96} = -1 \mod p \tag{44}$$

$$2^{64} = 2^{32} - 1 \mod p \tag{45}$$

Exponentiation

Max length N = p-1, highly composite, 2^{32}
How to find the primitive root of order p-1 ?
Program to search for primitive root
We find 7^5
Meaning that $7^{5(p-1)} = 1 \mod p$ for $N = p - 1$
For smaller DFT length of a divisor of $N - 1$, a generator is clearly $w = 7^{5(p-1)/N}$

13 A trier

But the most common usage of the DFT is by far the Complex field. Using complex numbers, the primitive N^{th} root of unity is defined by $w = e^{2\pi i/N}$. But here we will focus on the finite field of integers $\mathbb{Z}_p = \mathbb{Z}/p\mathbb{Z}$ where p is prime. Please note that the principles explained here can be extended to the ring $\mathbb{Z}_n$ (n not a prime) but that's not the focus of this document.

$\mathbb{Z}_p$ is nothing else but the set of integers from 0 to $p-1$, also called the set of residues because they are the results of calculations modulo p. Indeed all calculations will be made modulo p. This leads to the definition of the Number Theoretic Transform (NTT)

Definition 6. Number Theoretic Transform (NTT) in $\mathbb{Z}_p$
Let X be a sequence of N integers x_j all in the range from 0 to $p-1$.

$$X = \{x_0, x_1, ..., x_{N-1}\} \tag{46}$$

The NTT of X is the sequence $\widehat{X}$ defined by the following N elements, all integers

$$\widehat{x}_k = \sum_{j=0}^{N-1} x_j w^{jk} \mod p \tag{47}$$

where w is the primitive N-th root of unity in $\mathbb{Z}_p$, meaning that

$$\begin{cases} w^N = 1 \mod p \\ w^k \neq 1 \mod p \quad \text{for all } k \text{ between } 1 \text{ and } N-1 \end{cases} \tag{48}$$

So the usual complex root of unity w =exp(2 pi...) is simply replaced by w^N = $1 modp(citerBCAC08).Dansledocsurlesbases, prendrealorsunexemplesimple, parlerdesFNTetMNT.ParlerensuiteduCRT$ $x+1)$.

After collecting a large number of papers dedicated to FFT, it was clear that according to the popular opinion:

- FFT are complex and a good level of math is required to read academic papers
- After 50 years almost everything has been said on this topic
- NTT pure integer algorithms are presumably slow and cannot be easily used for very large convolutions
- Writing a extremely fast multiplication is a complex exercise reserved to very few people

Could some of the assumptions be false? Even though LucasNTT is a primality test used for the search of big Mersenne prime numbers, the interesting part of this document is about FFT, convolutions, NTT, fast multiplication and optimizations. The Lucas-Lehmer primality test is just here to illustrate and demonstrate the use and exactness of a fast multiplication algorithm.

```
__global__ void fft_CT(uint64_t* x, int stepNumber, int log_n, const uint64_t* __restrict__ roots)
{
        int tid = blockIdx.x * blockDim.x + threadIdx.x;
        int dist = 1 << (log_n - stepNumber);
        int idx = tid >> (log_n - stepNumber);
        int modIdx = tid & (dist - 1); // int modIdx = tid % dist;
        int Exp = modIdx << (stepNumber - 1);

        // Get the indices of the butterfly
        int a = modIdx + (idx << (log_n - stepNumber + 1));
        int b = a + dist;

        uint64_t u = x[a];
        uint64_t v = x[b];
        x[a] = add_Mod(u, v); // U = U + V
        x[b] = mul_Mod(sub_Mod(u, v), roots[Exp]); // V = W * (U - V)
}
```

14 Headings: first level

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula. See Section 14.

14.1 Headings: second level

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

$$\xi_{ij}(t) = P(x_t = i, x_{t+1} = j | y, v, w; \theta) = \frac{\alpha_i(t) a_{ij}^{w_t} \beta_j(t+1) b_j^{v_{t+1}}(y_{t+1})}{\sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i(t) a_{ij}^{w_t} \beta_j(t+1) b_j^{v_{t+1}}(y_{t+1})} \tag{49}$$

14.1.1 Headings: third level

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Paragraph Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

15 Examples of citations, figures, tables, references

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetuer at, consectetuer sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus. Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae, dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui. [**?** **?**] and see [**?**].

The documentation for `natbib` may be found at

http://mirrors.ctan.org/macros/latex/contrib/natbib/natnotes.pdf

Of note is the command `\citet`, which produces citations appropriate for use in inline text. For example,

```
\citet{hasselmo} investigated\dots
```

produces

Hasselmo, et al. (1995) investigated…

https://www.ctan.org/pkg/booktabs

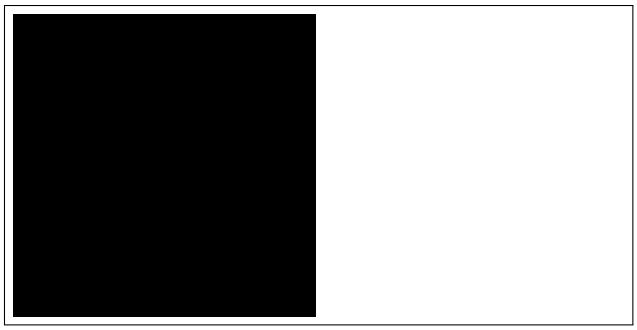

Figure 1: Sample figure caption.

Table 1: Sample table title

	Part	
Name	Description	Size (μm)
Dendrite	Input terminal	$\sim$100
Axon	Output terminal	$\sim$10
Soma	Cell body	up to 10^6

15.1 Figures

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi. See Figure 1. Here is how you add footnotes. [1] Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetuer eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

15.2 Tables

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetuer tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo. See awesome Table 1.

15.3 Lists

- Lorem ipsum dolor sit amet
- consectetur adipiscing elit.
- Aliquam dignissim blandit est, in dictum tortor gravida eget. In ac rutrum magna.

References

[1] W. M. Gentleman and G. Sande. Fast Fourier Transforms: for fun and profit. In *Proceedings of the November 7-10, 1966, fall joint computer conference on XX - AFIPS '66 (Fall)*, page 563, San Francisco, California, 1966. ACM Press.

[1] Sample of the first footnote.

[2] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–297, May 1965.

[3] R. C. Agarwal and J. W. Cooley. Fourier transform and convolution subroutines for the IBM 3090 Vector Facility. *IBM Journal of Research and Development*, 30(2):145–161, March 1986.

[4] Marshall C. Pease. An Adaptation of the Fast Fourier Transform for Parallel Processing. *J. ACM*, 15(2):252–264, April 1968.

[5] P. Duhamel, B. Piron, and J.M. Etcheto. On computing the inverse DFT. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 36(2):285–286, February 1988.

[6] Irving S. Reed. The Use of Finite Fields and Rings to Compute Convolutions:. Technical report, Defense Technical Information Center, Fort Belvoir, VA, June 1975.

[7] Richard E Crandall. Integer convolution via split-radix fast galois transform. *Center for Advanced Computation Reed College*, 1999.

[8] I. Reed and Treiu-Kien Truong. Convolutions over residue classes of quadratic integers. *IEEE Transactions on Information Theory*, 22(4):468–475, July 1976.

[9] Dubois and Venetsanopoulos. The Discrete Fourier Transform Over Finite Rings with Application to Fast Convolution. *IEEE Transactions on Computers*, C-27(7):586–593, July 1978.

[10] Richard Crandall and Barry Fagin. Discrete weighted transforms and large-integer arithmetic. *Mathematics of Computation*, 62(205):305–305, January 1994.